

Data Structure

Course No. 40989

Week #11

Red-Black Tree & Priority Queue

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430

(Re) Red-Black Tree

What is a Red-Black Tree?

Definition

- A binary search tree in which every node is colored with either **Red** or **Black**
- It satisfies the **Red-Black properties**

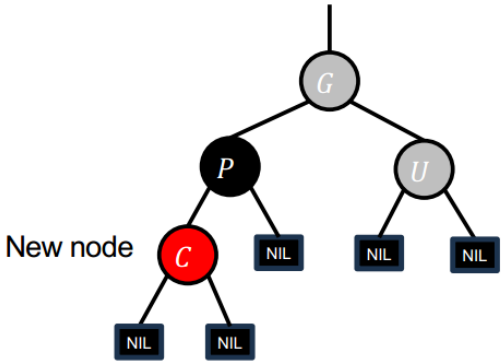
Red-Black Properties

- Every node is either **Red** or **Black**.
- The root is always **Black**.
- Every leaf (NULL=NIL) is **Black**.
- If a node is **Red**, both its children are **Black** (a **Black** node can have both **Red**/**Black** children).
- For each node, all paths from a node A to a NIL node B contain the same number of **Black** nodes.

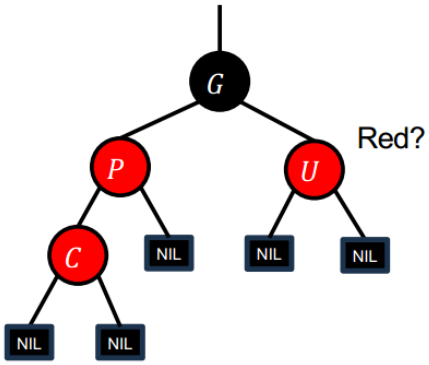
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Insertion

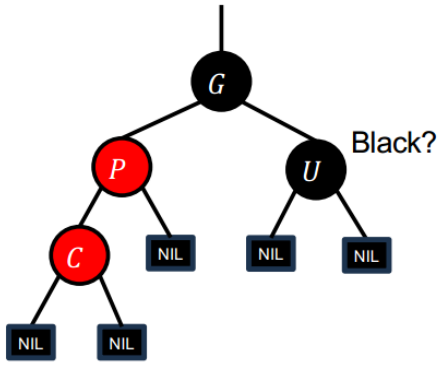
- Inserting a key w into T
  - [1] Perform standard BST insertion and make the color of new node as **Red**
  - [2] If parent is **Black**, then do nothing!
  - [3] If parent is **Red**, then do the following:
    - [3-a] If 'uncle' node U is **Red**, then **'Recolor'**
    - [3-b] If 'uncle' node U is **Black**, then **'Rotate+ Recolor'**



(2): the parent is black



(3)-(a): the uncle is red



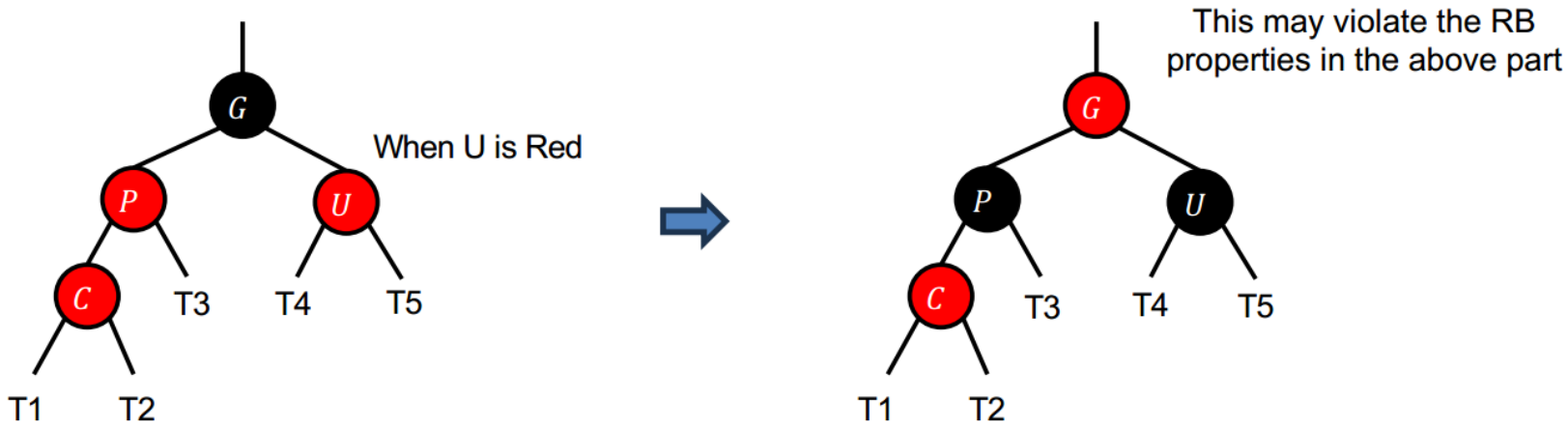
(3)-(b): the uncle is black

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Insertion

[CASE 1] U is **RED** : Recoloring

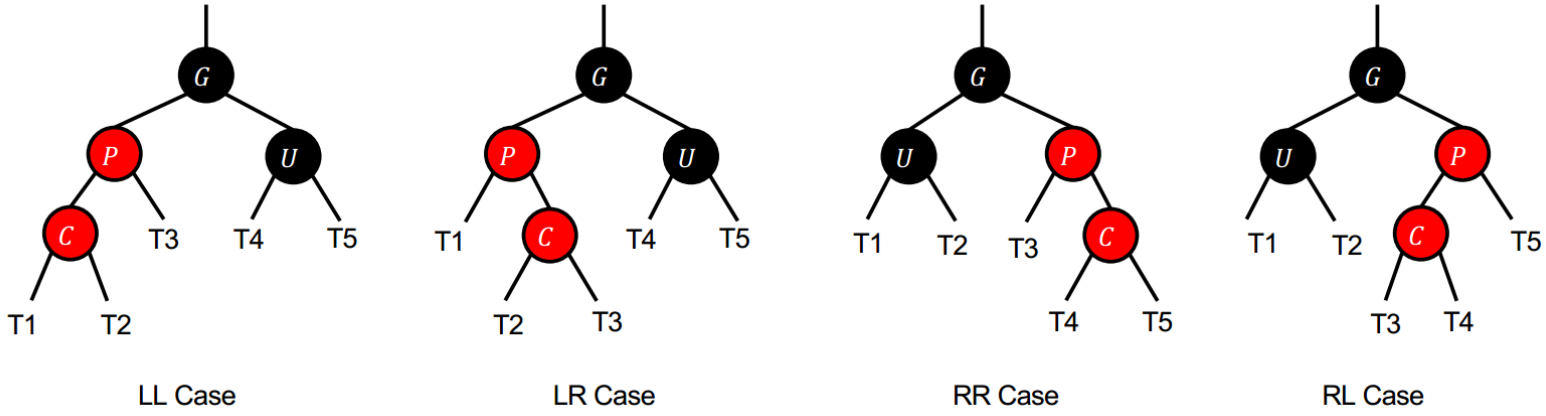
- [1] Change the colors of the parent P and the uncle U to **Black**
- [2] Change color of grandparent G as **Red**
- [3] Treat G as the New Node and repeat [2] and [3]



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Insertion

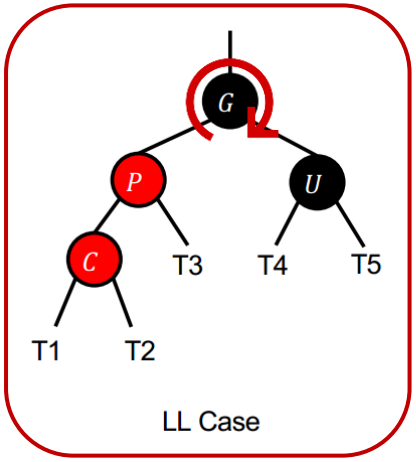
- [CASE 2] U is **Black** : Rotation + Recoloring
  - [1] Preserve the property of Binary Search Tree (left subtree <= root <= right subtree)
  - **[2] Rotate according to four scenarios : LL, LR, RR, RL**

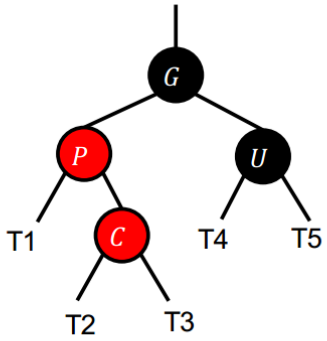


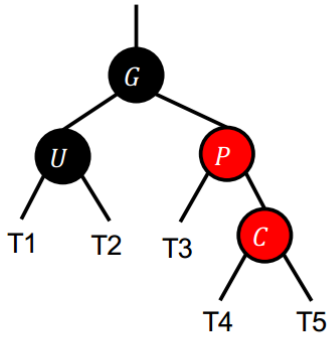
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

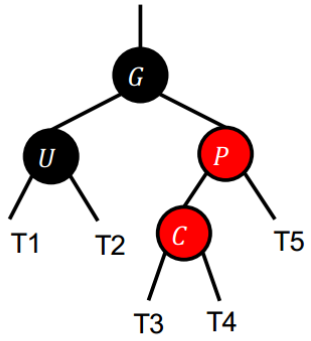
Operations: Insertion

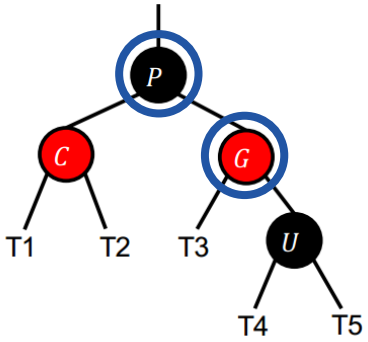
- [CASE 2] U is **Black** : Rotation + Recoloring
  - [1] Preserve the property of Binary Search Tree (left subtree <= root <= right subtree)
  - **[2] Rotate according to four scenarios : LL, LR, RR, RL**







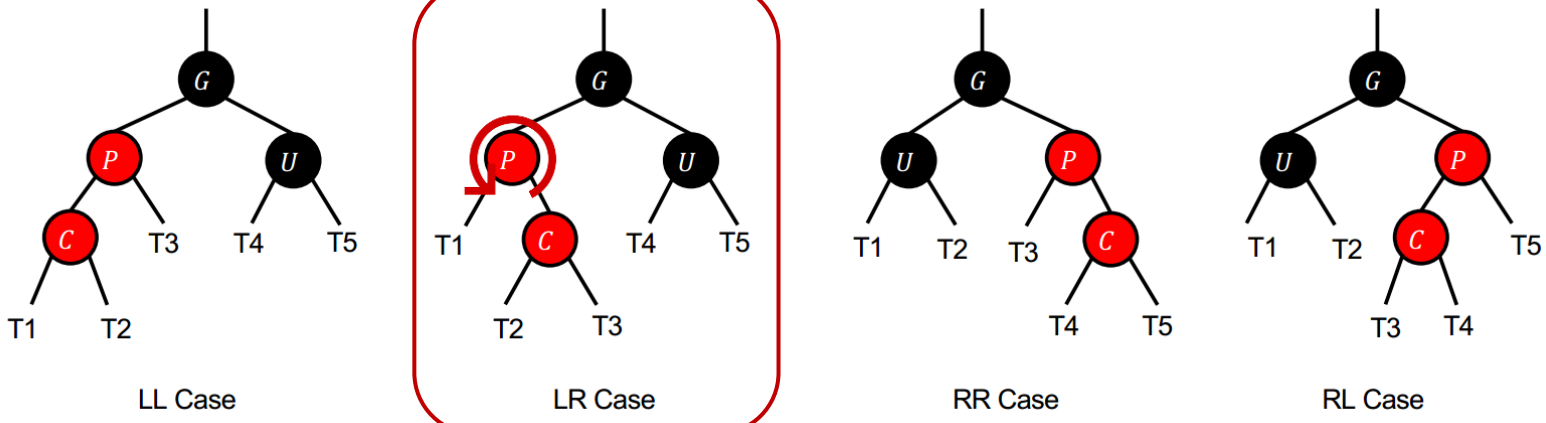


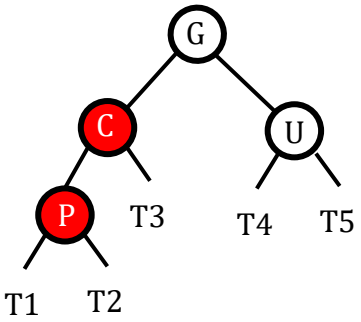


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Insertion

- [CASE 2] U is **Black** : Rotation + Recoloring
  - [1] Preserve the property of Binary Search Tree (left subtree <= root <= right subtree)
  - **[2] Rotate according to four scenarios : LL, LR, RR, RL**

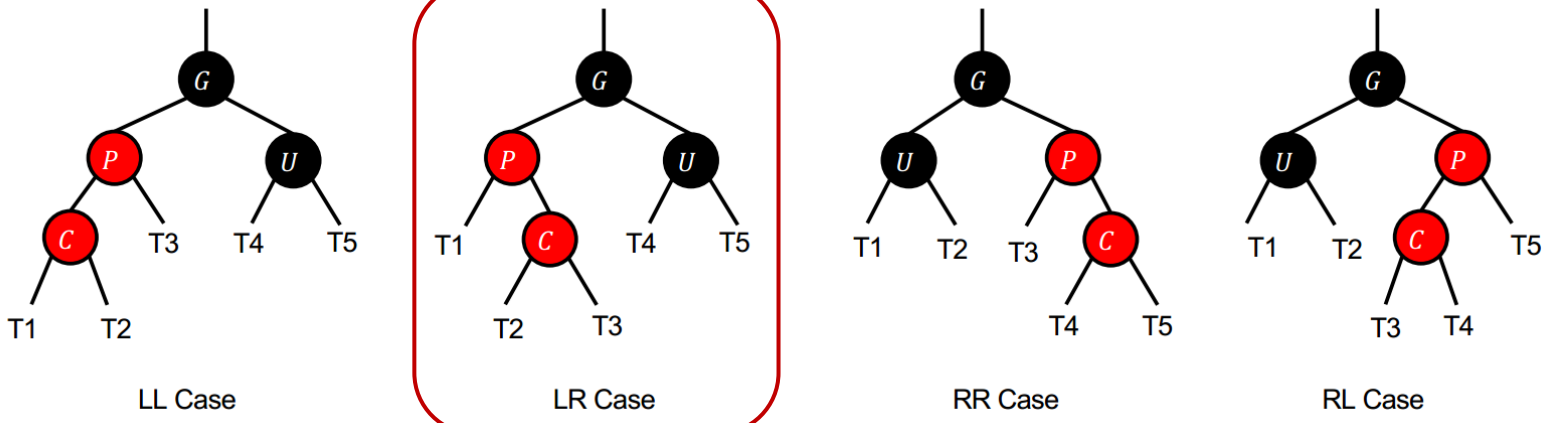


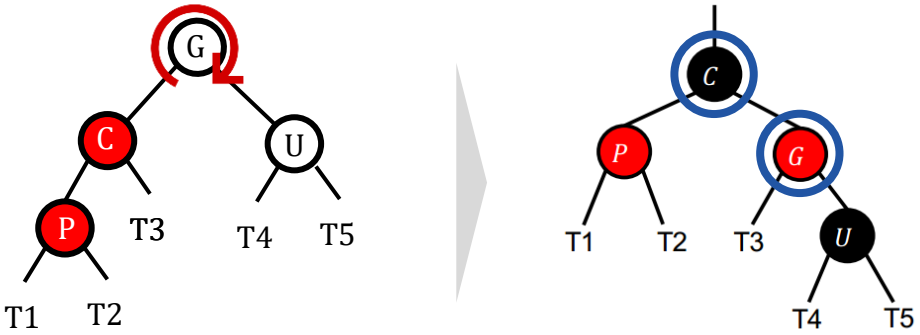


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Insertion

- [CASE 2] U is **Black** : Rotation + Recoloring
  - [1] Preserve the property of Binary Search Tree (left subtree <= root <= right subtree)
  - **[2] Rotate according to four scenarios : LL, LR, RR, RL**

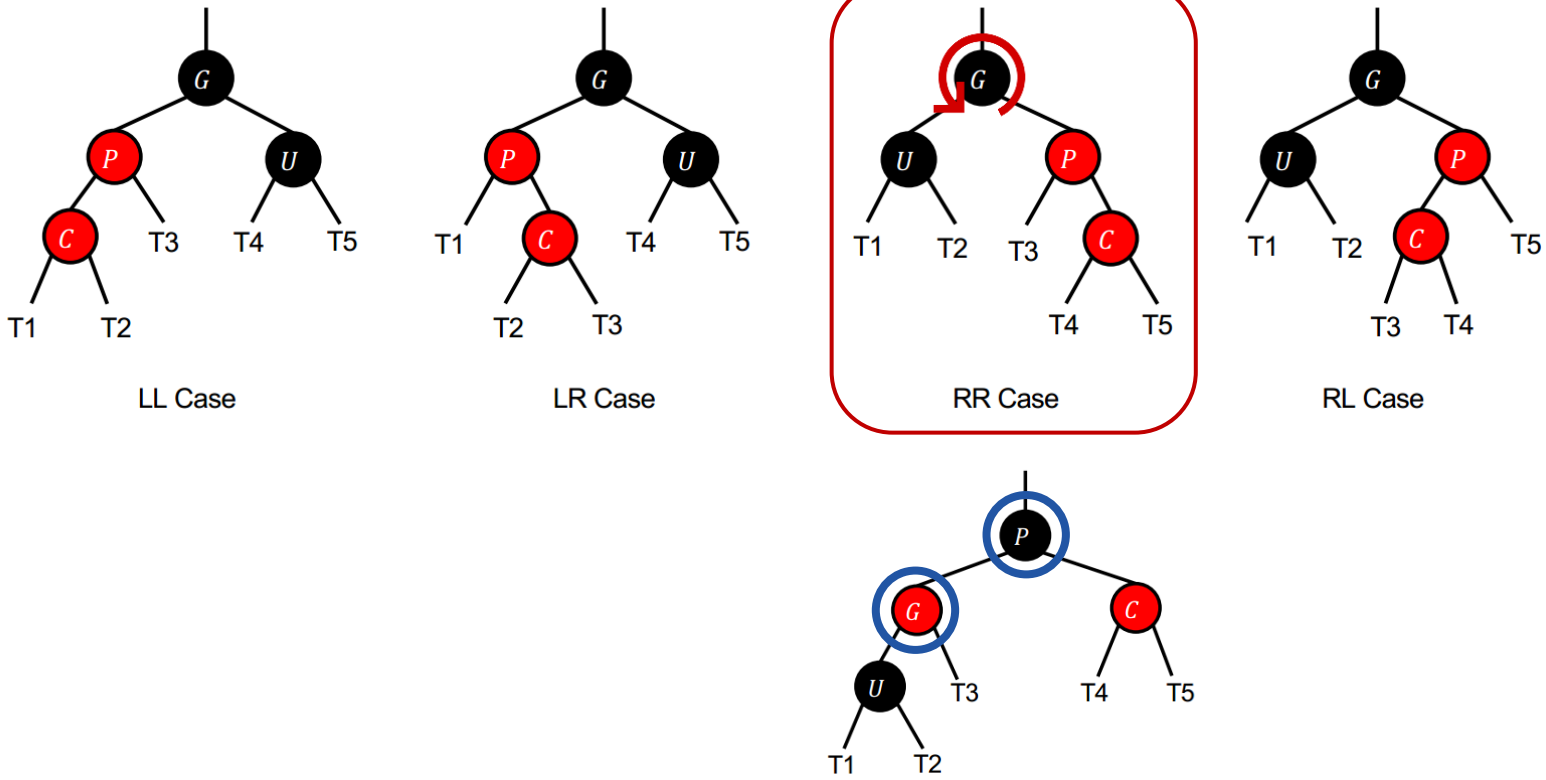




Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Insertion

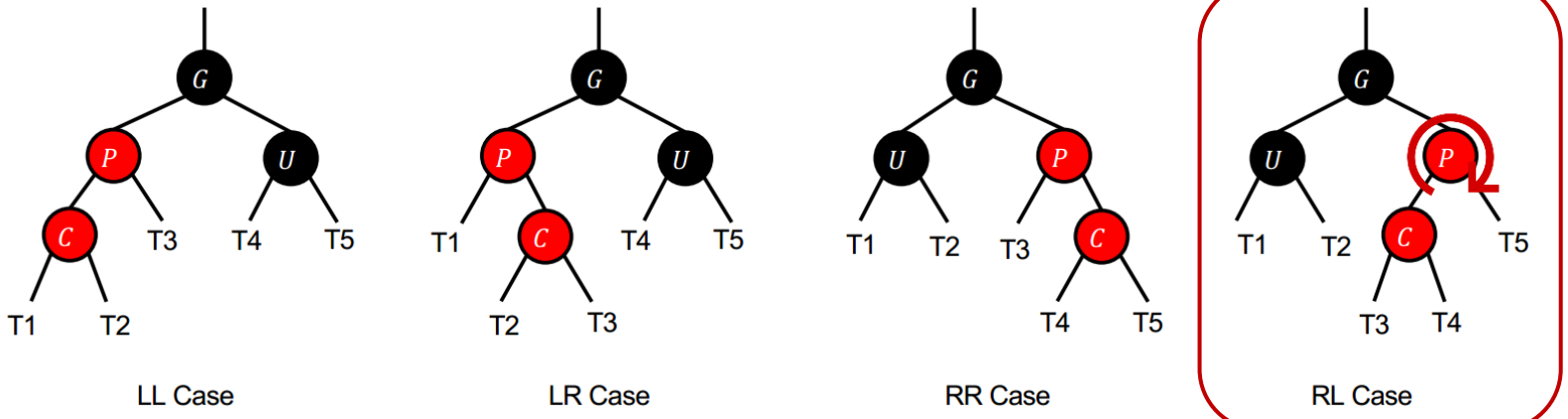
- [CASE 2] U is **Black** : Rotation + Recoloring
  - [1] Preserve the property of Binary Search Tree (left subtree <= root <= right subtree)
  - **[2] Rotate according to four scenarios : LL, LR, RR, RL**

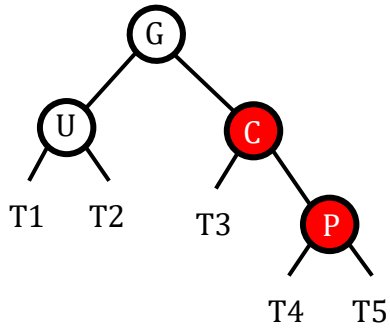


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Insertion

- [CASE 2] U is **Black** : Rotation + Recoloring
  - [1] Preserve the property of Binary Search Tree (left subtree <= root <= right subtree)
  - **[2] Rotate according to four scenarios : LL, LR, RR, RL**

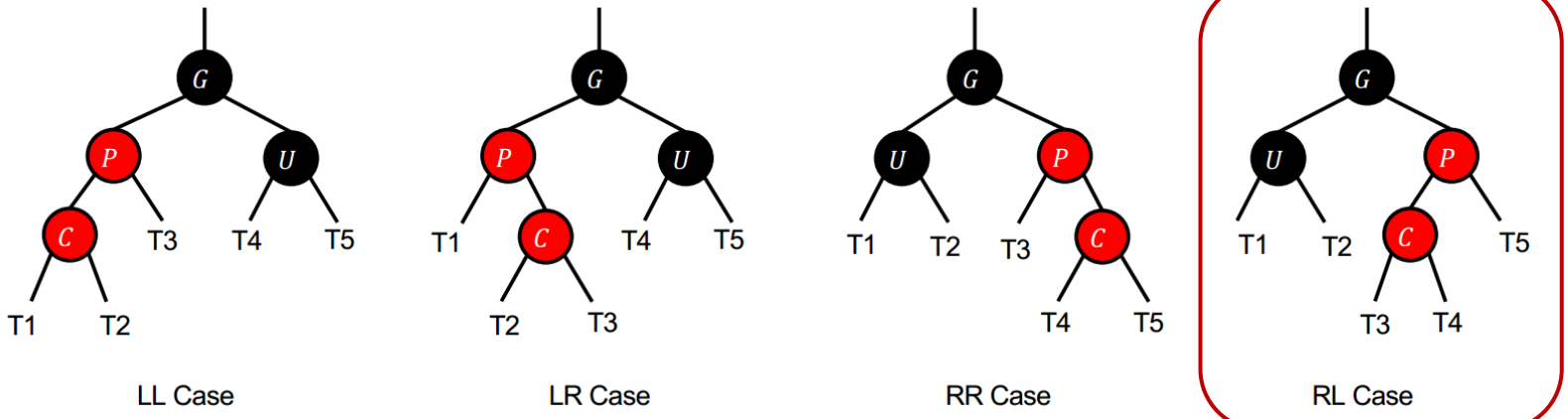


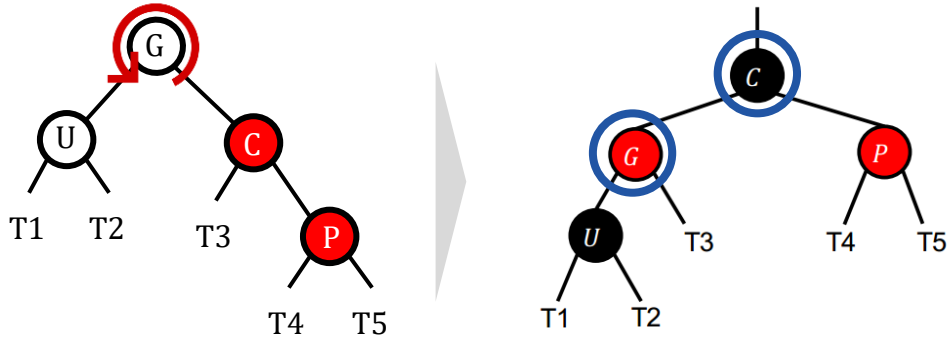


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Insertion

- [CASE 2] U is **Black** : Rotation + Recoloring
  - [1] Preserve the property of Binary Search Tree (left subtree <= root <= right subtree)
  - **[2] Rotate according to four scenarios : LL, LR, RR, RL**





Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

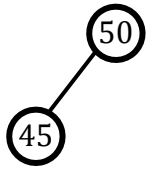
- Worst case in BST



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

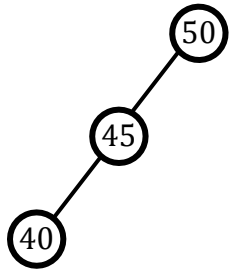
- Worst case in BST



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

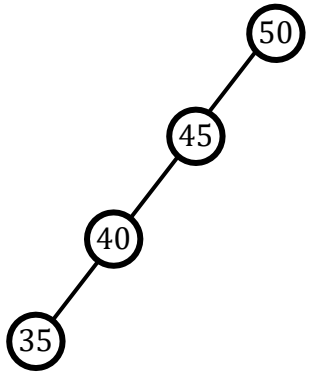
- Worst case in BST



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

- Worst case in BST



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

- Worst case in BST





Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

- Worst case in BST

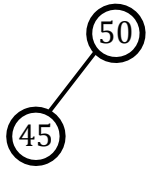


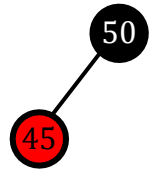


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

- Worst case in BST

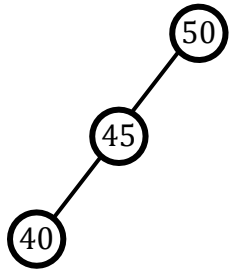


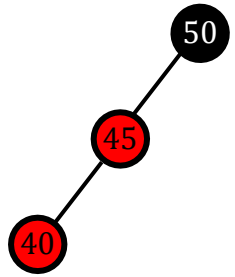


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

- Worst case in BST

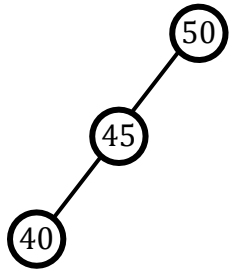


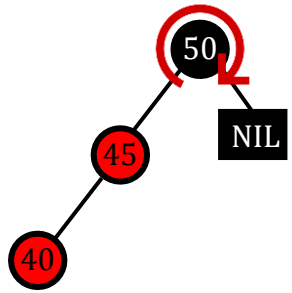


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

- Worst case in BST

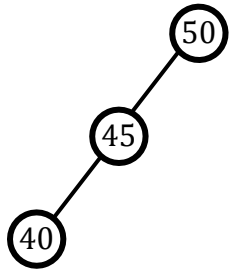


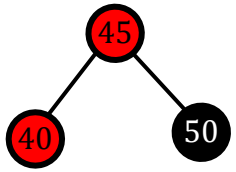


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

- Worst case in BST



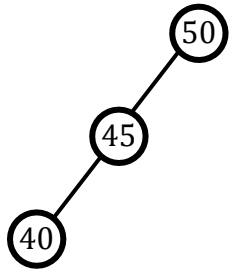


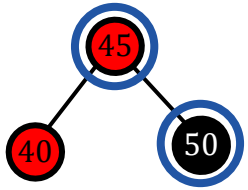
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Red-Black Tree

Why Red-Black Tree?

- Worst case in BST

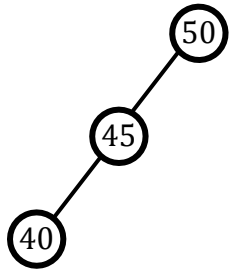


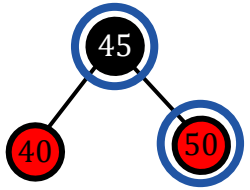


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

- Worst case in BST



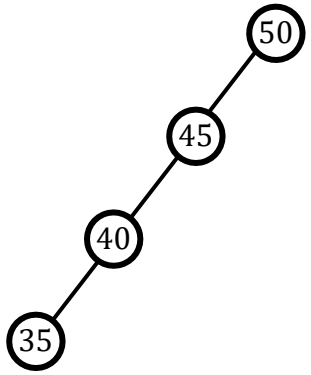


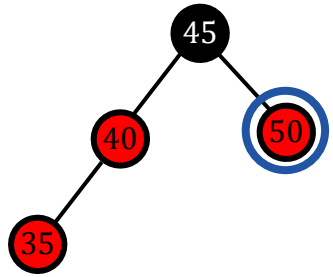
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Red-Black Tree

Why Red-Black Tree?

- Worst case in BST

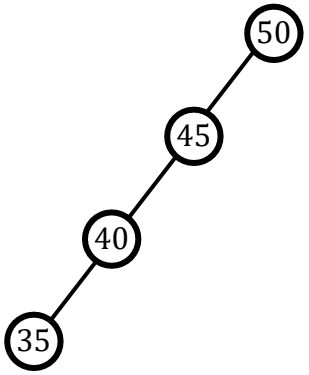


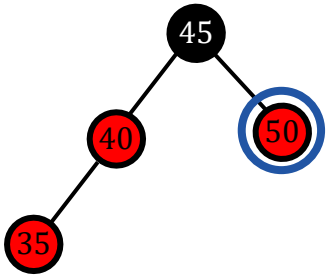


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Why Red-Black Tree?

- Worst case in BST



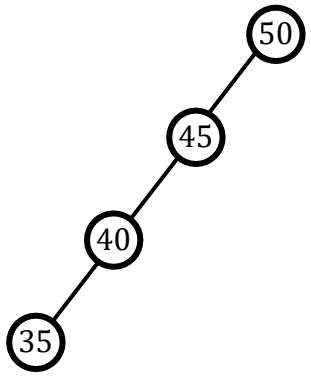


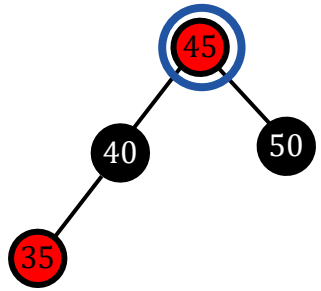
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Red-Black Tree

Why Red-Black Tree?

- Worst case in BST



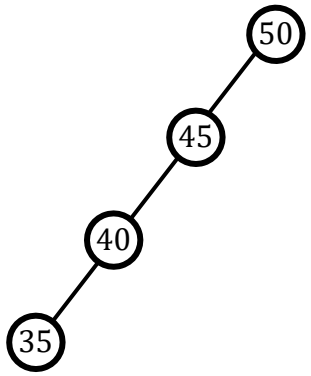


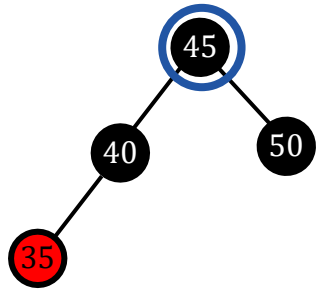
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Red-Black Tree

Why Red-Black Tree?

- Worst case in BST

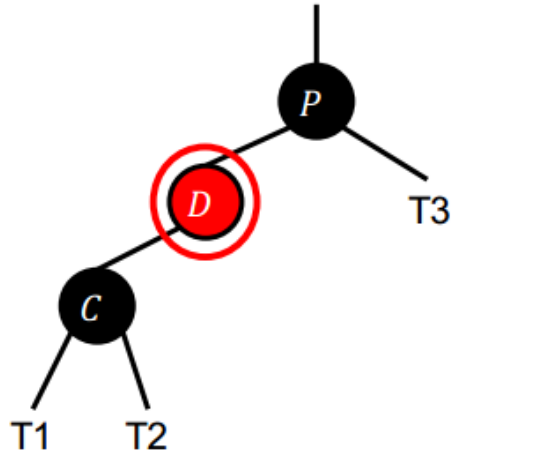




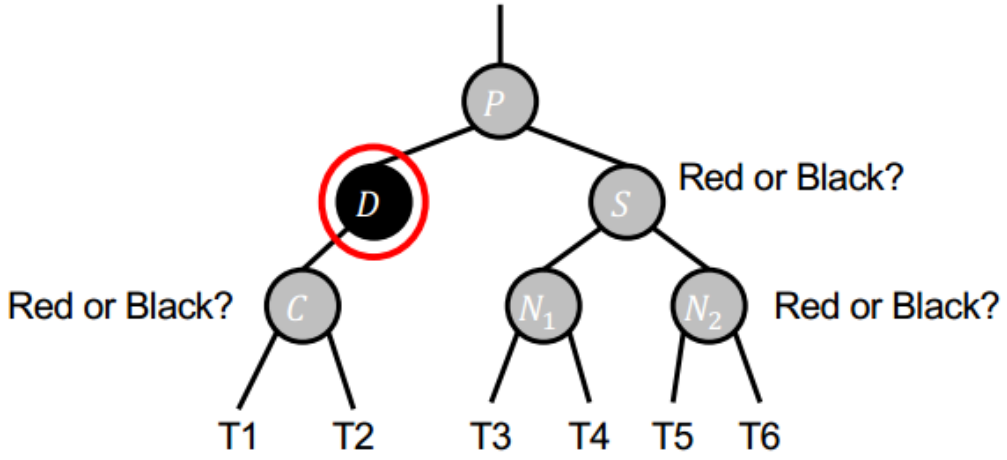
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Deletion

- Deleting D from BST T
  - [1] If the node has no child? Simply remove it!
  - [2] If the node has a single child? Replace the node with its child!
  - [3] If the node has two children? Replace the node with its successor!



(2): the node to be deleted is Red



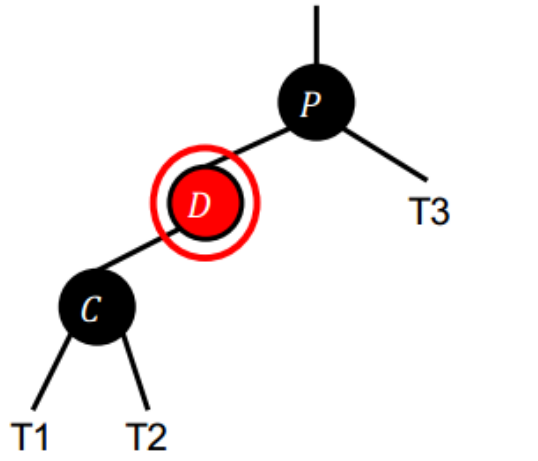
(3) The node to be deleted is Black

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

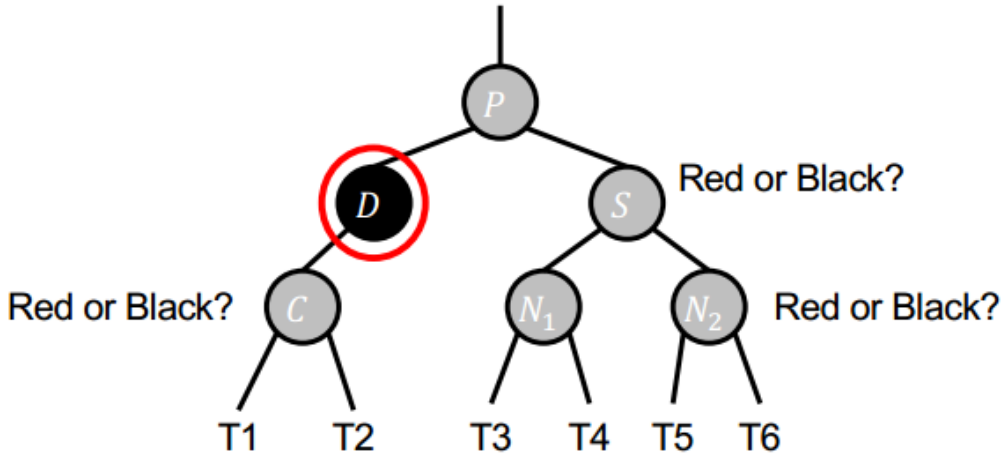
Operations: Deletion

- Deleting **Red** D from Red-Black Tree T
  - [1] If the node has no child? Simply remove it!
  - [2] If the node has a single child? Replace the node with its child!
  - [3] If the node has two children? Replace the node with its successor!

No violation if D is **Red**



(2): the node to be deleted is Red

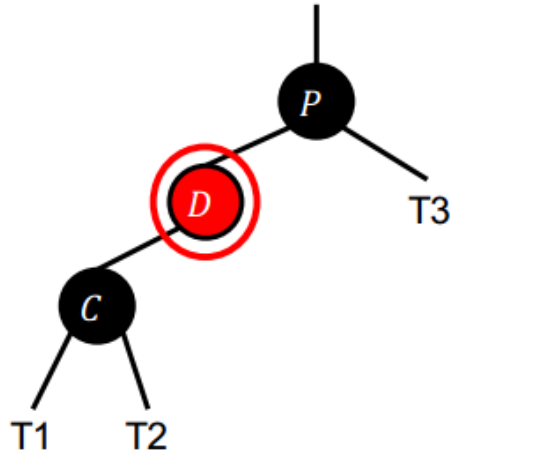


(3) The node to be deleted is Black

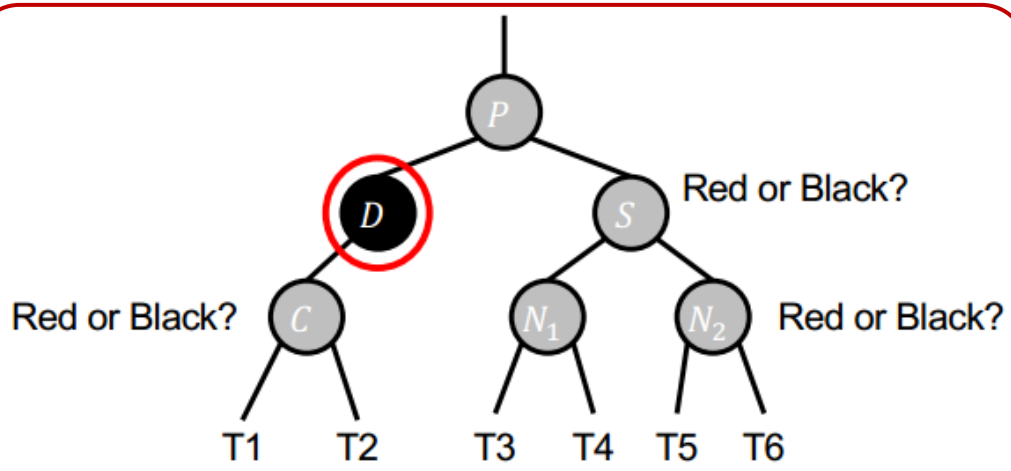
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Deletion

- Deleting **Black** D from Red-Black Tree T
  - [CASE 1] Successor C is Red
  - [CASE 2] Successor C is Black



(2): the node to be deleted is Red



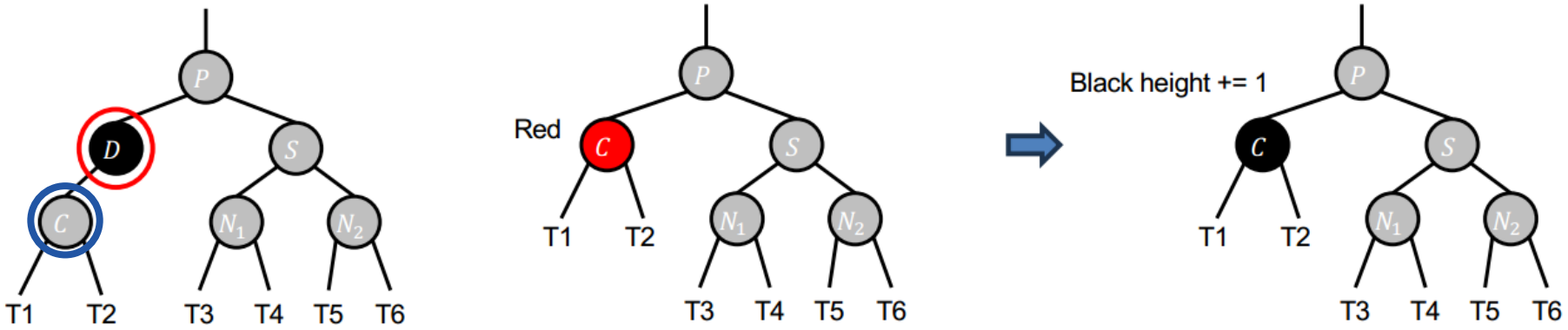
(3) The node to be deleted is Black

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Deletion

- Deleting **Black** D from Red-Black Tree T
  - **[CASE 1] Successor C is Red**
  - [CASE 2] Successor C is Black

- **Solution: Recoloring**
  - Change the color of node C as **Black**

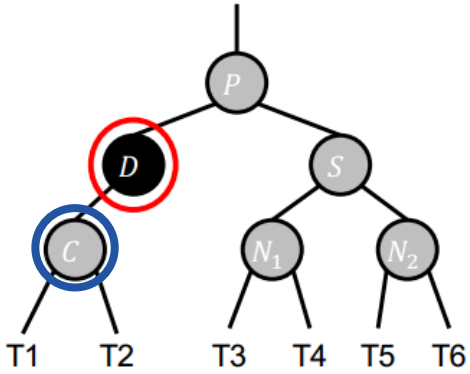


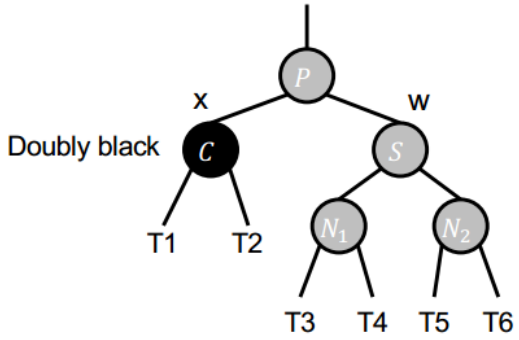
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

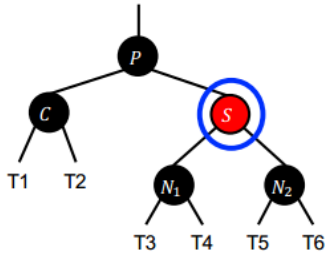
Operations: Deletion

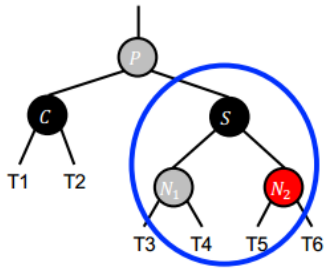
- Deleting **Black** D from Red-Black Tree T
  - [CASE 1] Successor C is Red
  - **[CASE 2] Successor C is Black**

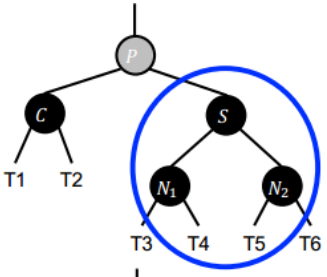
- **Solution: Rotation + Recoloring**
  - Four cases

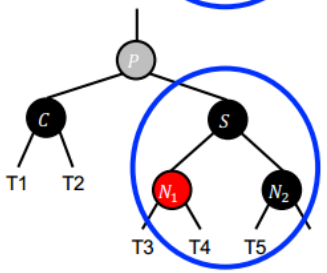










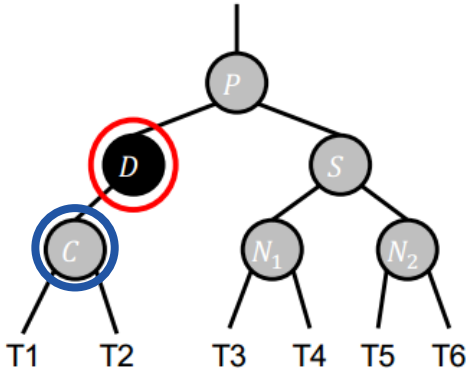


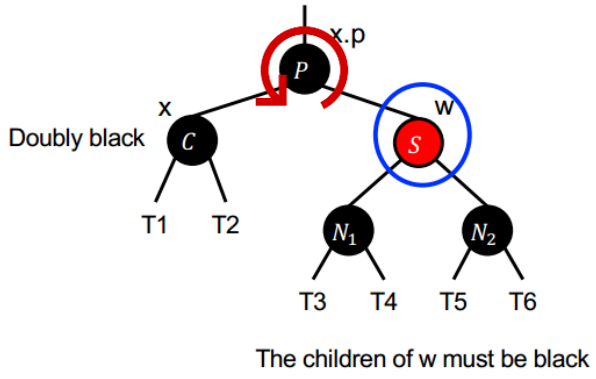
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Deletion

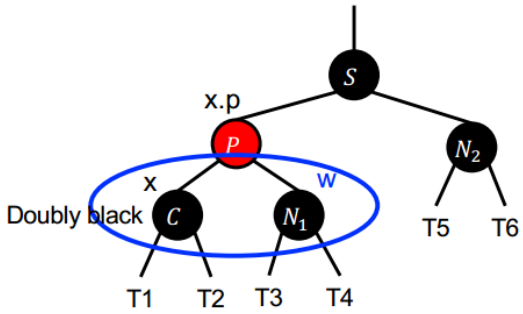
- Deleting **Black** D from Red-Black Tree T
  - [CASE 1] Successor C is Red
  - **[CASE 2] Successor C is Black**

- **[CASE 2-1] Sibling is Red**
  - Solution: Left Rotation + Recoloring





The children of w must be black



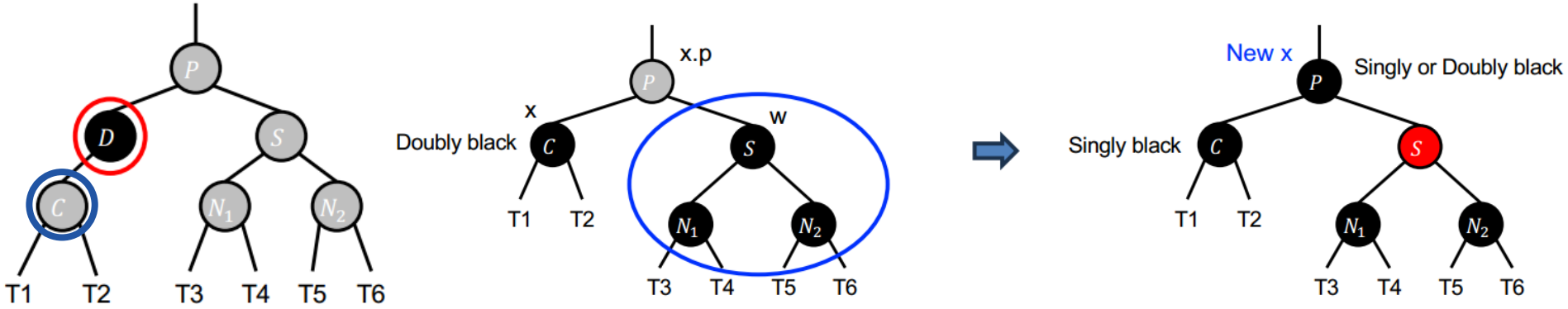
Convert Case 2-1 to into Case 2-2, 2-3, or 2-4.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Deletion

- Deleting **Black** D from Red-Black Tree T
  - [CASE 1] Successor C is Red
  - **[CASE 2] Successor C is Black**

- **[CASE 2-2] Sibling is Black + children are Black**
  - Solution: Recoloring

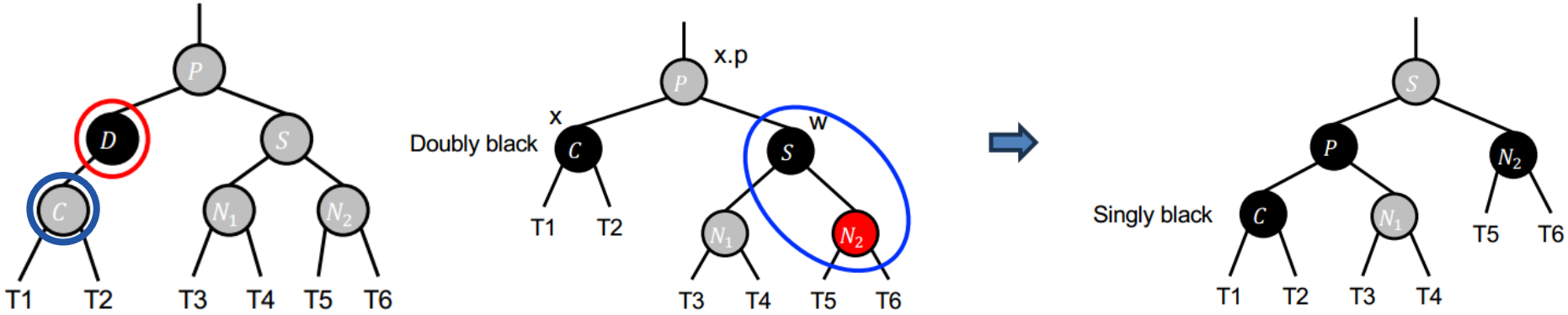


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Deletion

- Deleting **Black** *D* from Red-Black Tree *T*
  - [CASE 1] Successor C is Red
  - **[CASE 2] Successor C is Black**

- **[CASE 2-3] Sibling is Black + right child is Red**
  - Solution: Left Rotation + Recoloring (P and N2 to **Black**)

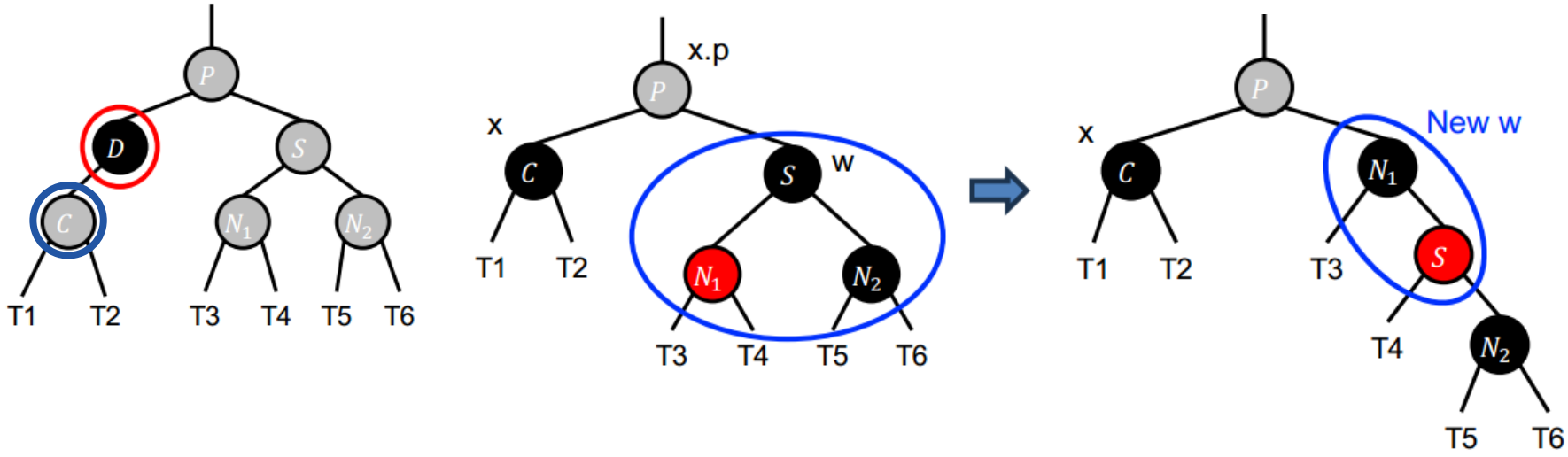


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Deletion

- Deleting **Black** *D* from Red-Black Tree *T*
  - [CASE 1] Successor C is Red
  - **[CASE 2] Successor C is Black**

- **[CASE 2-4] Sibling is Black + left child is Red + right child is Black**
  - Solution: Right Rotation + Recoloring (N1 to **Black, S to Red**) + Left Rotation + Recoloring (P to **Black**, S to **Black**)

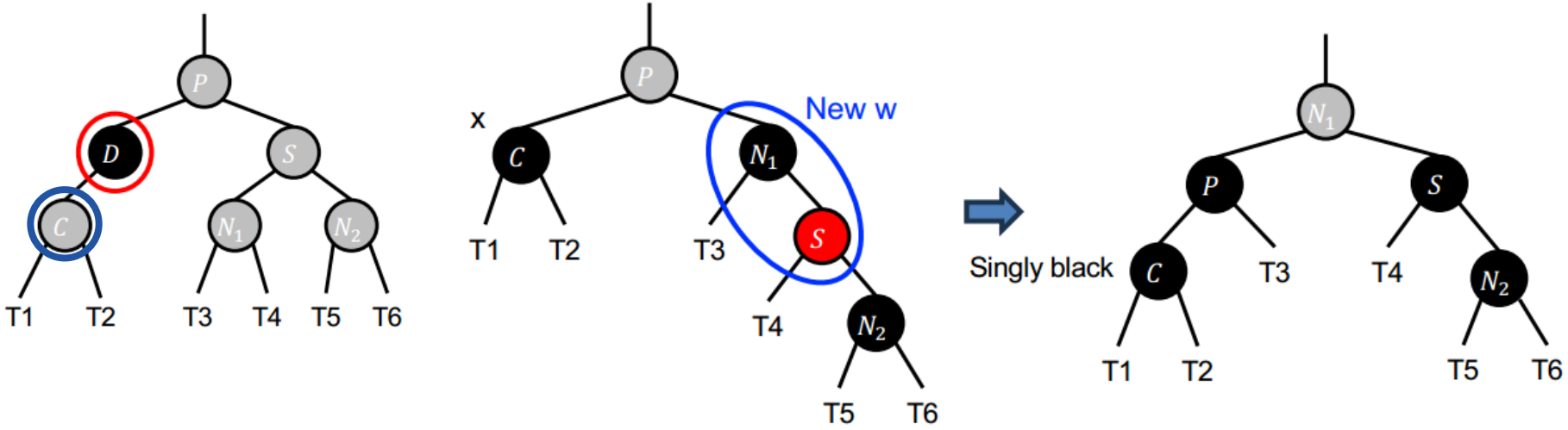


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operations: Deletion

- Deleting **Black** D from Red-Black Tree T
  - [CASE 1] Successor C is Red
  - **[CASE 2] Successor C is Black**

- **[CASE 2-4] Sibling is Black + left child is Red + right child is Black**
  - Solution: Right Rotation + Recoloring (N1 to **Black, S to Red**) + Left Rotation + Recoloring (P to **Black**, S to **Black**)



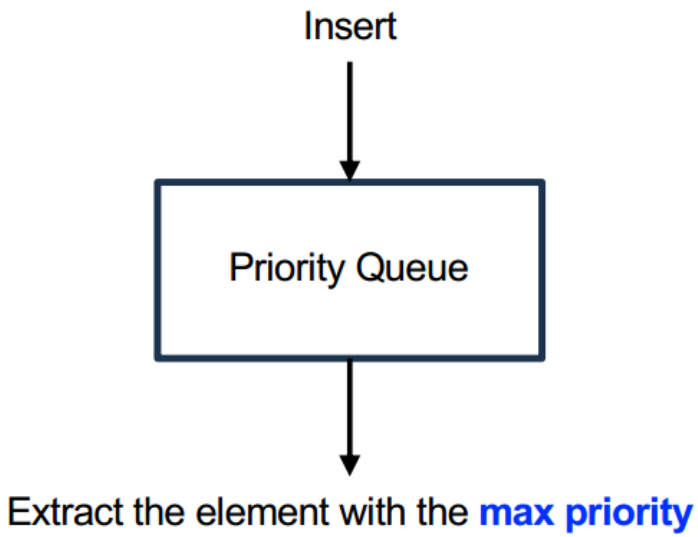
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Priority Queue

What is a Priority Queue?

Definition

- A type of queue that arranges elements based on their priority
- Key operations: Insert(x, Q), getMax(Q), getMin(Q), deleteMax(Q), deleteMin(Q)



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Priority Queue

What is a Priority Queue?

vs Ordinary Queue

- Queue : First-In-First-Out (FIFO)
- Priority Queue : Order in elements inserted != Order in elements processed



Hey, Bumsoo! Good morning!

- We need to submit this contract until 11:30am!
- Oh and I hope you didn't forget that by the end of the week you need to submit your monthly report.
- Our lunch will be a team meeting, so reserve a restaurant that could accommodate 11 people.
- And the request from the financial department should be taken care of before 10am, take care of this first.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Priority Queue

What is a Priority Queue?

vs Ordinary Queue

- Queue : First-In-First-Out (FIFO)
- Priority Queue : Order in elements inserted != Order in elements processed



Hey, Bumsoo! Good morning!

- We need to submit this contract until 11:30am!
- Oh and I hope you didn't forget that by the end of the week you need to submit your monthly report.
- Our lunch will be a team meeting, so reserve a restaurant that could accommodate 11 people.
- And the request from the financial department should be taken care of before 10am, take care of this first.

- [1] Submit contract
- [2] Submit monthly report.
- [3] reserve restaurant.
- [4] take care of request from financial department.



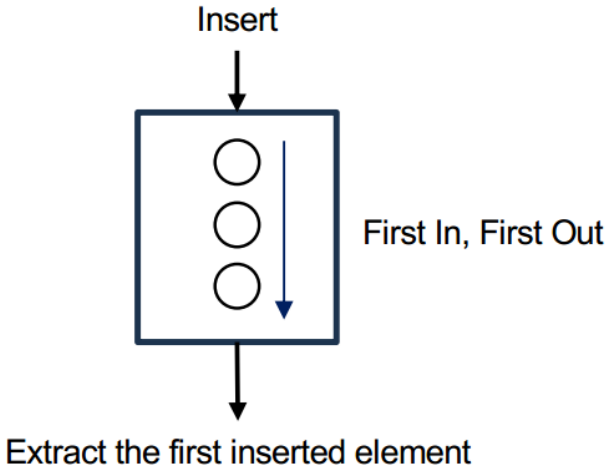
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

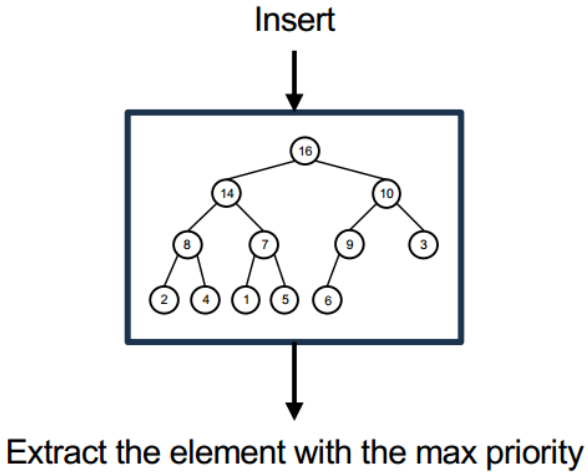
Priority Queue

What is a Priority Queue?

vs Ordinary Queue

- Queue : First-In-First-Out (FIFO)
- Priority Queue : Order in elements inserted != Order in elements processed





Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Priority Queue

Implementation

- Priority Queue can be implemented with various data structures
  - Array (ordered / unordered)
  - Linked List
  - **Binary Heap**
  - …

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

What is a Binary Heap?

- A Binary Heap is a Complete Binary Tree that satisfies the Heap Property

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

What is a Binary Heap?

- A Binary Heap is a **Complete Binary Tree** that satisfies the Heap Property

Complete Binary Tree

- A binary tree that has all levels completely filled with nodes except the last level.
- At the last level, all the nodes are as far left as possible.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

What is a Binary Heap?

- A Binary Heap is a Complete Binary Tree that satisfies the **Heap Property**

Heap Property

- Heap is a Binary Tree that satisfies the Heap Property
- Max Heap: The parent is greater or equal to its child
- Min-Heap: The parent is less than equal to its child

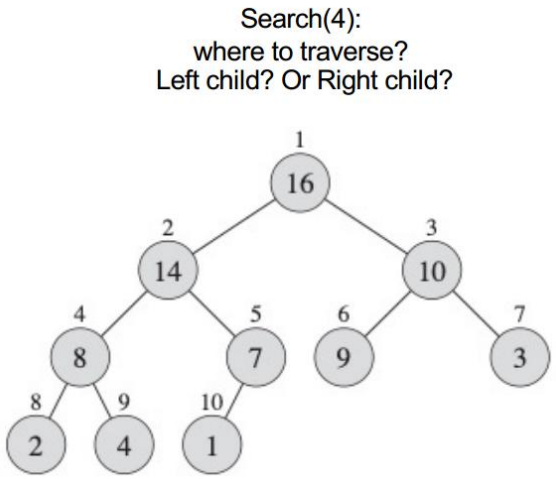
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

Heap Property

- Heap is a Binary Tree that satisfies the Heap Property
- Max Heap: The parent is greater or equal to its child
- Min-Heap: The parent is less than equal to its child

- The heap property may indicate that the elements are sorted in a certain order, but it is **useless when it comes to searching**



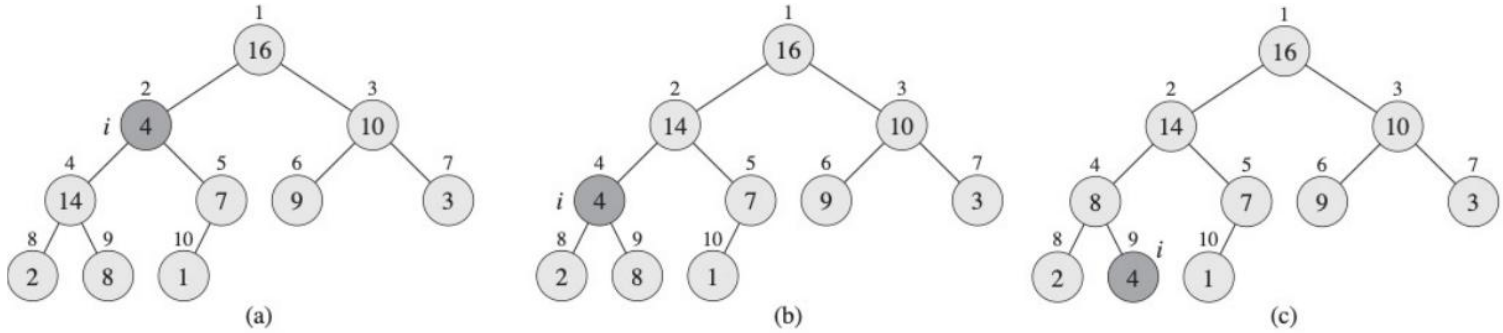
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

Max-Heapify

- Heap is a Binary Tree that satisfies the Heap Property
- Max Heap: The parent is greater or equal to its child
- Min-Heap: The parent is less than equal to its child

```
MAX-HEAPIFY(A, i)
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4       largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7       largest = r
8  if largest ≠ i
9       exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```



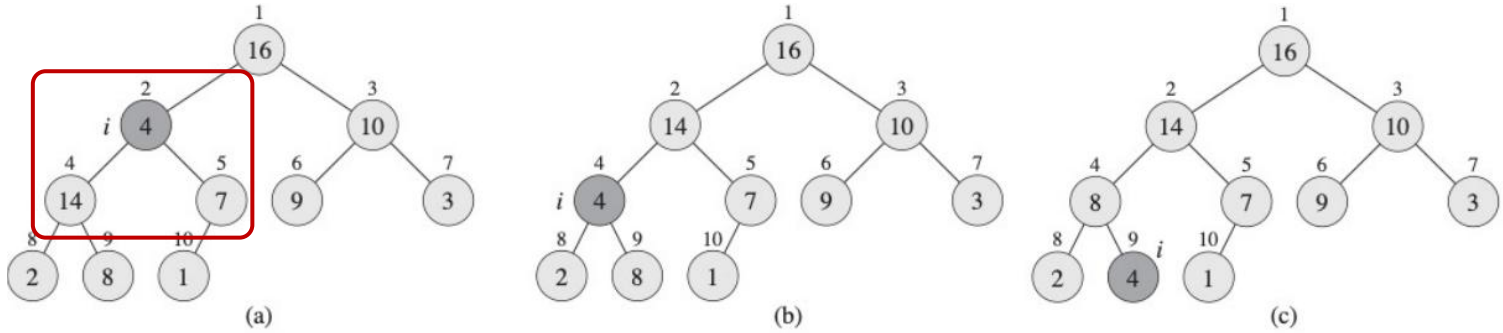
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

Max-Heapify

- Heap is a Binary Tree that satisfies the Heap Property
- Max Heap: The parent is greater or equal to its child
- Min-Heap: The parent is less than equal to its child

```
MAX-HEAPIFY(A, i)
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4       largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7       largest = r
8  if largest ≠ i
9       exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```



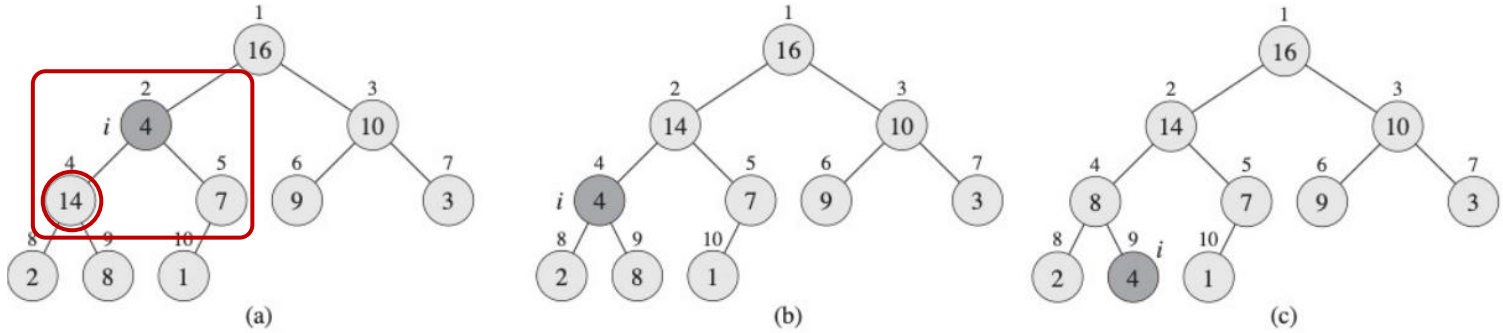
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

Max-Heapify

- Heap is a Binary Tree that satisfies the Heap Property
- Max Heap: The parent is greater or equal to its child
- Min-Heap: The parent is less than equal to its child

```
MAX-HEAPIFY(A, i)
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4       largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7       largest = r
8  if largest ≠ i
9       exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```



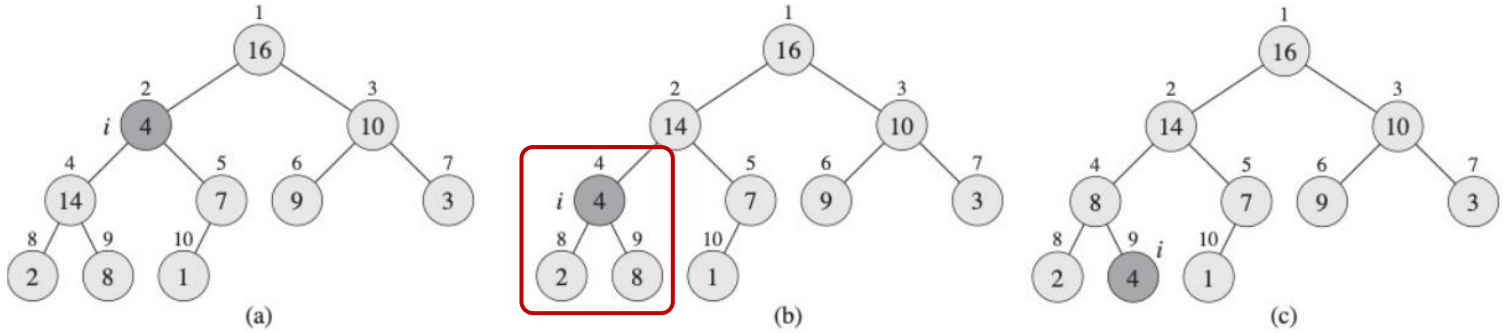
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

Max-Heapify

- Heap is a Binary Tree that satisfies the Heap Property
- Max Heap: The parent is greater or equal to its child
- Min-Heap: The parent is less than equal to its child

```
MAX-HEAPIFY(A, i)
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4       largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7       largest = r
8  if largest ≠ i
9       exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```



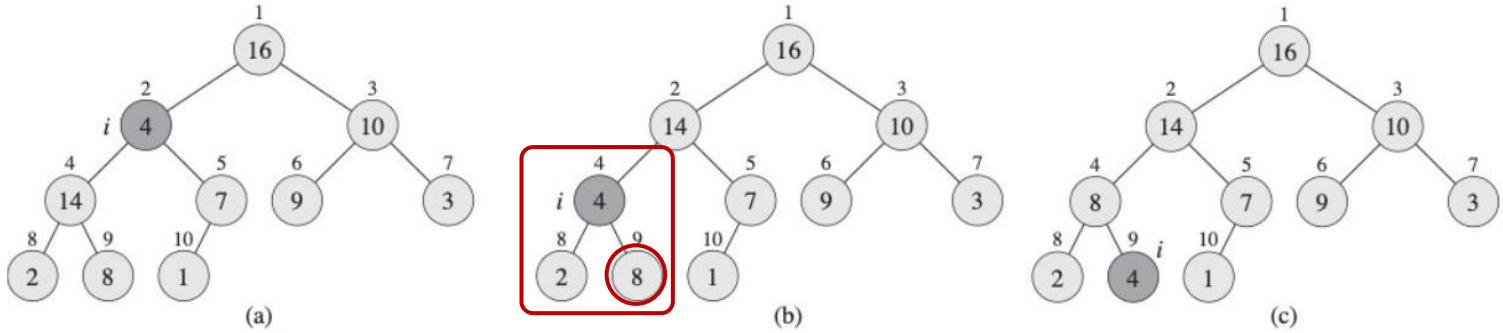
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

Max-Heapify

- Heap is a Binary Tree that satisfies the Heap Property
- Max Heap: The parent is greater or equal to its child
- Min-Heap: The parent is less than equal to its child

```
MAX-HEAPIFY(A, i)
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4       largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7       largest = r
8  if largest ≠ i
9       exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

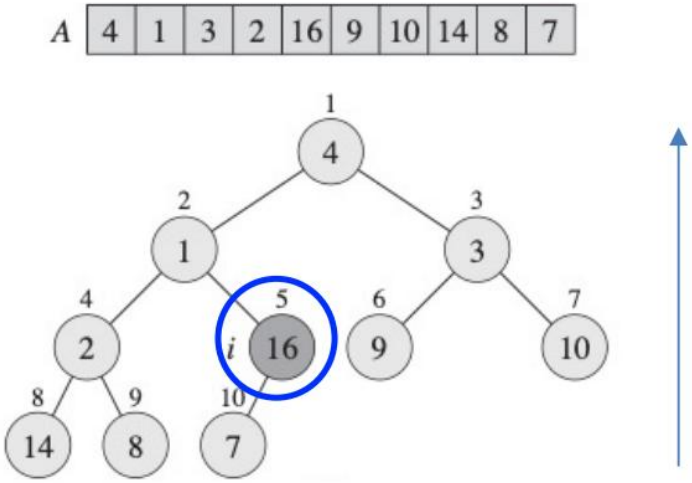
Implementation: Binary Heap

Building a Max Binary Heap

- Since the Binary Heap is a complete binary tree, it can be written as an array
- Start from the leaf node and move up

```
BUILD-MAX-HEAP(A)
1  A.heap-size = A.length
2  for i = ⌊A.length/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

- Let say we have the elements written in random order



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

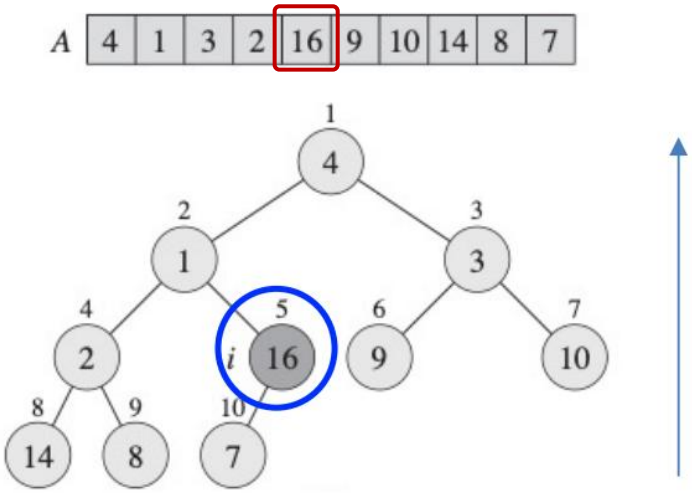
Implementation: Binary Heap

Building a Max Binary Heap

- Since the Binary Heap is a complete binary tree, it can be written as an array
- Start from the leaf node and move up

```
BUILD-MAX-HEAP(A)
1  A.heap-size = A.length
2  for i = ⌊A.length/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

- Let say we have the elements written in random order



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

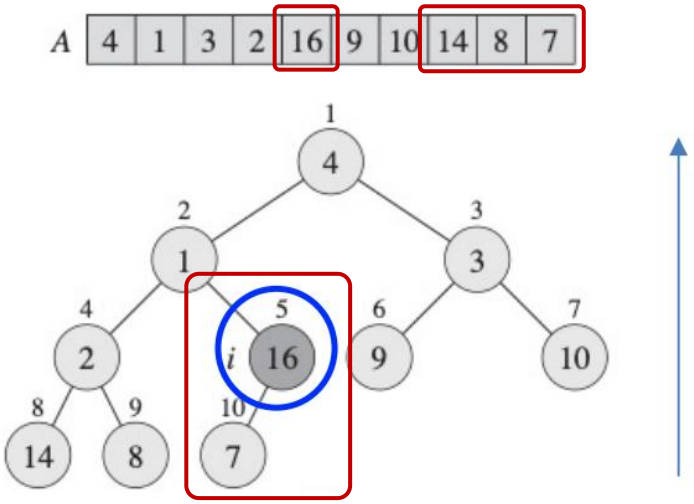
Implementation: Binary Heap

Building a Max Binary Heap

- Since the Binary Heap is a complete binary tree, it can be written as an array
- Start from the leaf node and move up

```
BUILD-MAX-HEAP(A)
1  A.heap-size = A.length
2  for i = ⌊A.length/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

- Let say we have the elements written in random order



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

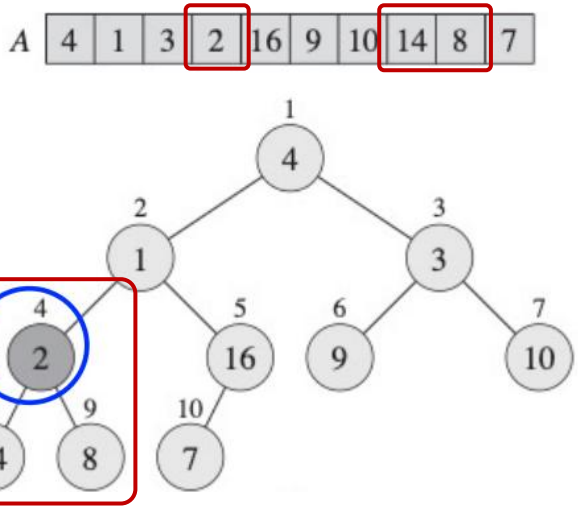
Implementation: Binary Heap

Building a Max Binary Heap

- Since the Binary Heap is a complete binary tree, it can be written as an array
- Start from the leaf node and move up

```
BUILD-MAX-HEAP(A)
1  A.heap-size = A.length
2  for i = ⌊A.length/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

- Let say we have the elements written in random order



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

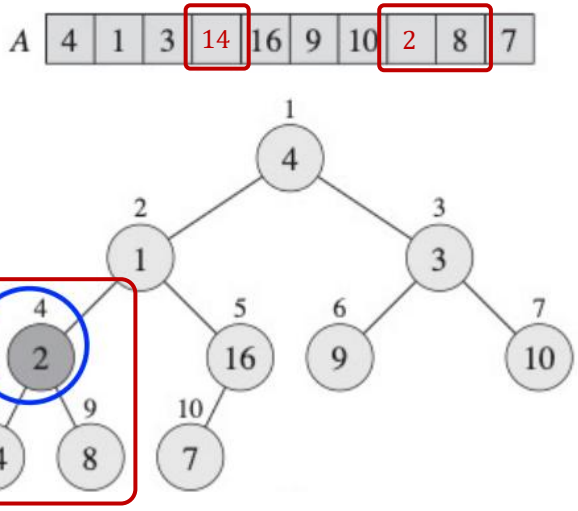
Implementation: Binary Heap

Building a Max Binary Heap

- Since the Binary Heap is a complete binary tree, it can be written as an array
- Start from the leaf node and move up

```
BUILD-MAX-HEAP(A)
1  A.heap-size = A.length
2  for i = ⌊A.length/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

- Let say we have the elements written in random order



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

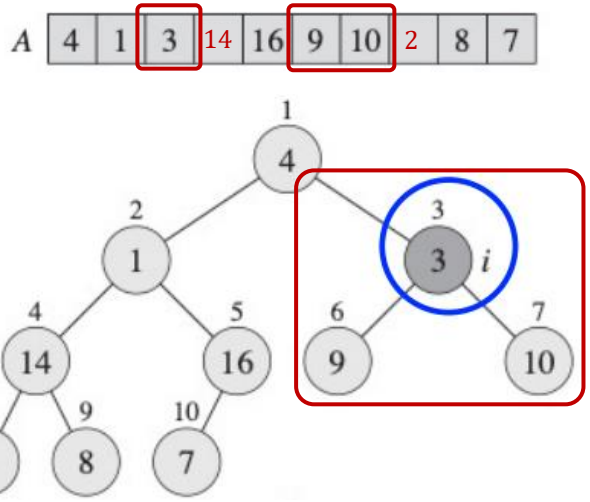
Implementation: Binary Heap

Building a Max Binary Heap

- Since the Binary Heap is a complete binary tree, it can be written as an array
- Start from the leaf node and move up

```
BUILD-MAX-HEAP(A)
1  A.heap-size = A.length
2  for i = ⌊A.length/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

- Let say we have the elements written in random order

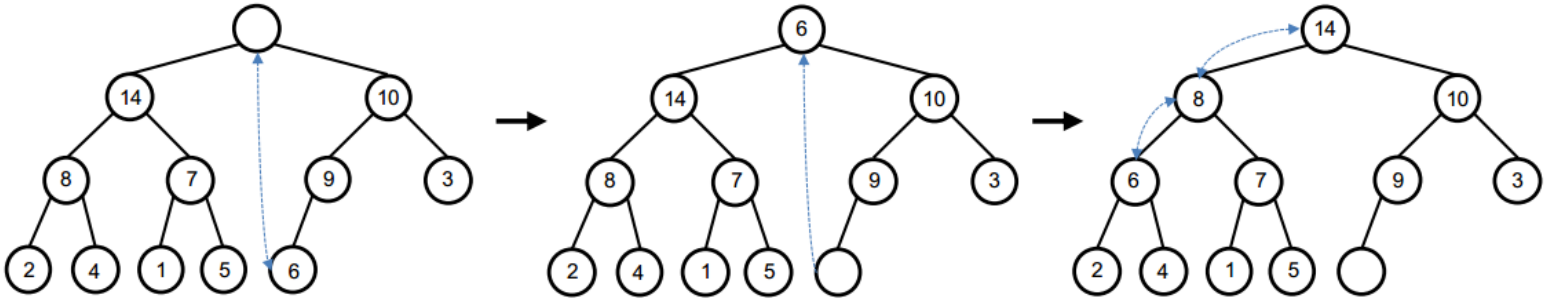


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

Extract Max (Deletion)

- Extract Element with the highest priority
- Reorganize the priority queue using Max-Heapify



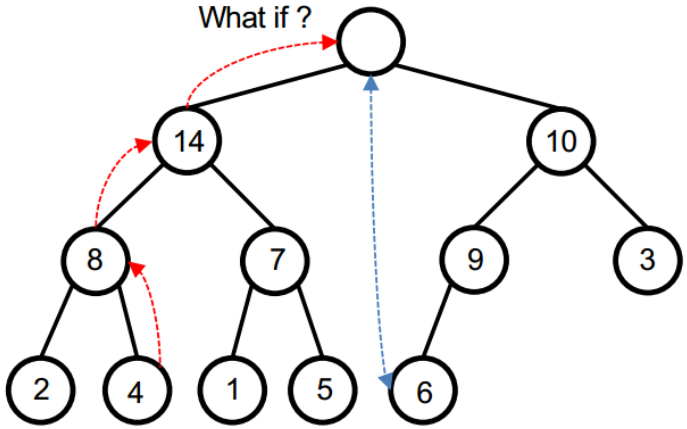
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

Extract Max (Deletion)

- Extract Element with the highest priority
- Reorganize the priority queue using Max-Heapify

- Why don't we move the larger child node upward repeatedly?



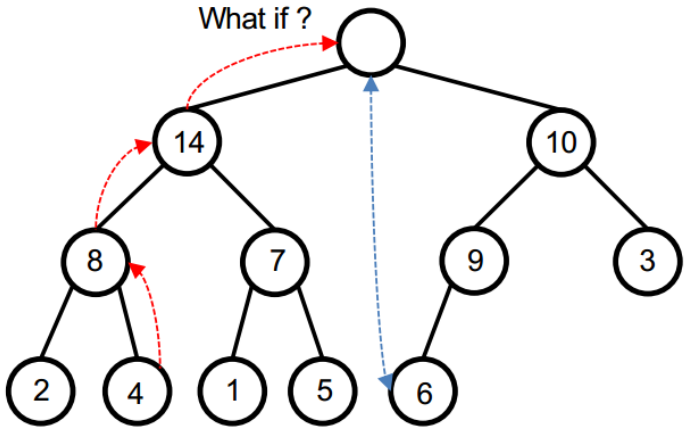
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

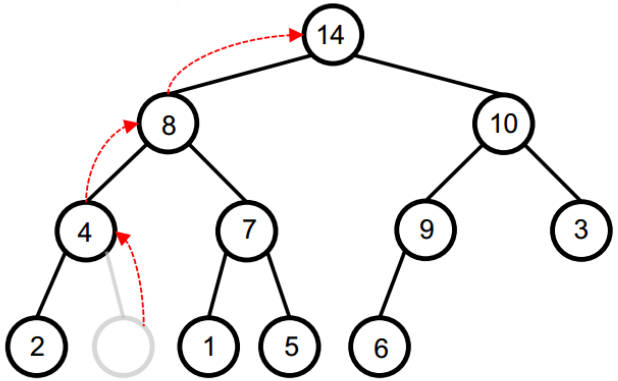
Implementation: Binary Heap

Extract Max (Deletion)

- Extract Element with the highest priority
- Reorganize the priority queue using Max-Heapify

- Why don't we move the larger child node upward repeatedly?





It is not a complete binary tree (heap) anymore!

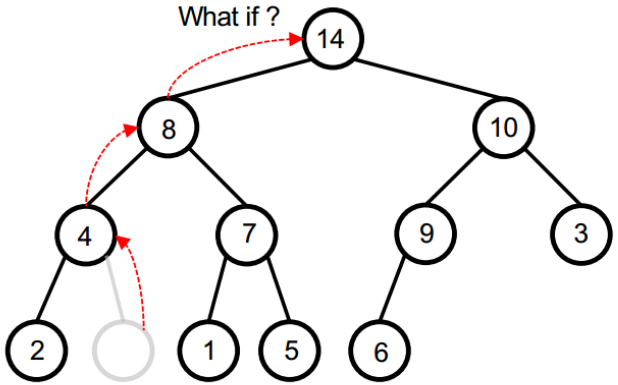
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

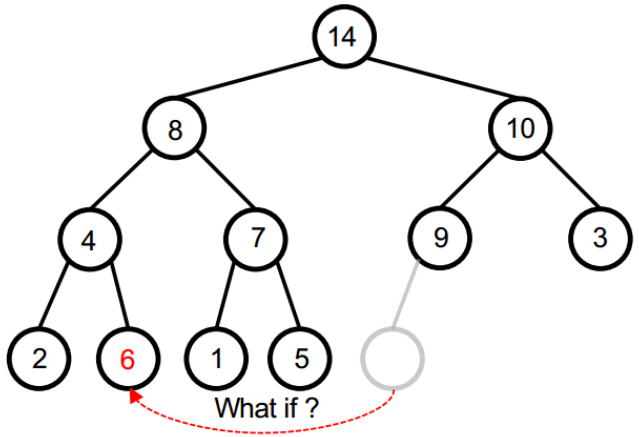
Extract Max (Deletion)

- Extract Element with the highest priority
- Reorganize the priority queue using Max-Heapify

- What if we move the last node to the empty space?



It is not a complete binary tree (heap) anymore!



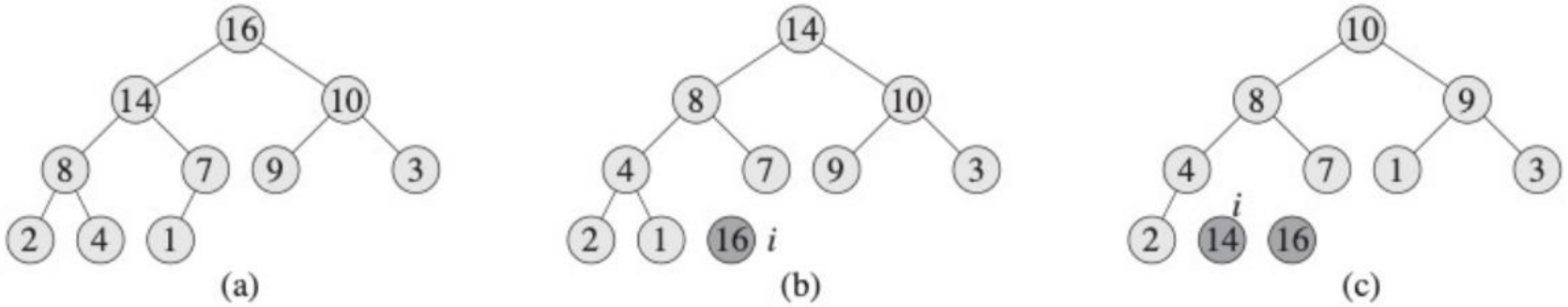
It may violate the max-heap property!

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

HeapSort

- [1] Build a Max-Heap
- [2] Exchange the maximum element at the root A[1] with A[i]
- [3] Call Max-Heapify (decreasing its size by 1)

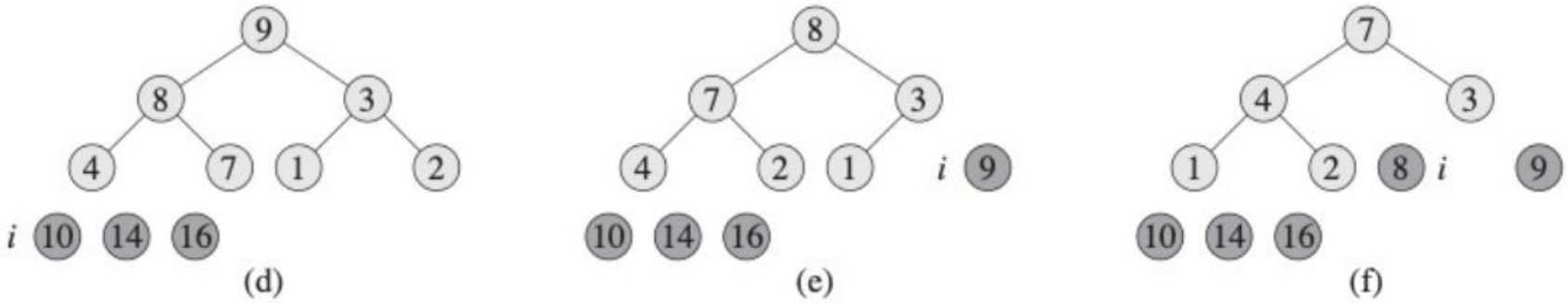


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

HeapSort

- [1] Build a Max-Heap
- [2] Exchange the maximum element at the root A[1] with A[i]
- [3] Call Max-Heapify (decreasing its size by 1)

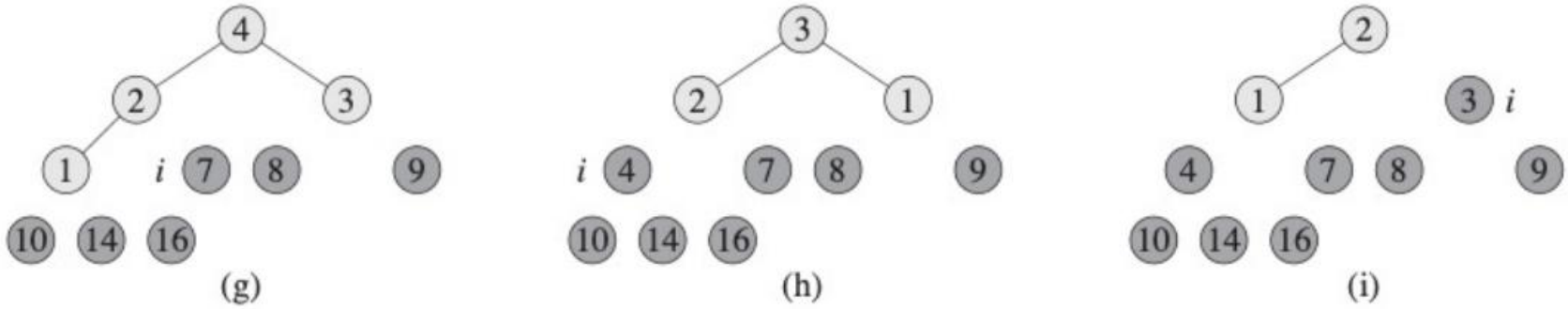


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

HeapSort

- [1] Build a Max-Heap
- [2] Exchange the maximum element at the root A[1] with A[i]
- [3] Call Max-Heapify (decreasing its size by 1)

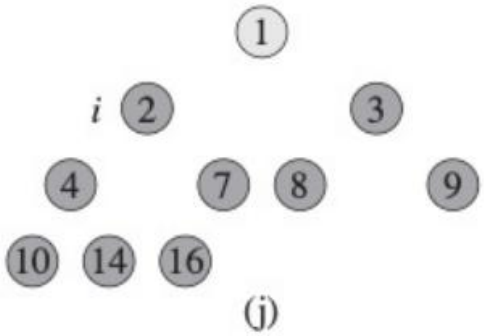


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Implementation: Binary Heap

HeapSort

- [1] Build a Max-Heap
- [2] Exchange the maximum element at the root A[1] with A[i]
- [3] Call Max-Heapify (decreasing its size by 1)



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential



Thank you

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430